

Atividade Prática 1 — Classificador Bayesiano

Etapa 4 — Resultados dos Testes e Reflexão Crítica

Autor	Marcus Vinícius Santos de Almeida
Disciplina	Mineração de Dados
Data de entrega	04/09/2026
Cobre os entregáveis	"Resultados dos testes" (5+ casos com análise) e "Reflexão crítica"
Artefatos	<code>testes/casos_teste.csv</code> · <code>testes/rodar_casos_teste.py</code> · <code>testes/resultados_casos.txt</code>
Modelo de treino	1.000.000 de módulos sintéticos da Etapa 2 (decisão "Naive Bayes puro" — features independentes entre si) · $P(\text{SIM}) = 350.000/1.000.000 = 0,3500$ · $P(\text{NAO}) = 650.000/1.000.000 = 0,6500$

Todos os números deste relatório saem de `python3 testes/rodar_casos_teste.py` (que reconstrói o banco DuckDB, roda os 6 casos e calcula os log-odds a partir da view `verossimilhancas` da Etapa 3), rodado sobre a massa de dados da versão atual da Etapa 2 — 6 features geradas independentes entre si, agora com $N = 1.000.000$ de registros (ver `CLAUDE.md`, seções "Naive Bayes puro" e "Migração para DuckDB e $N = 1.000.000$ registros", e o relatório da Etapa 2, §1.2). Os números desta versão do relatório são a terceira revisão da massa de dados do projeto (dados correlacionados → $N=150$ independente → $N=1.000.000$ independente) — como já documentado nas versões anteriores, cada mudança de dados muda os números, não a lógica do classificador nem a tese da Reflexão Crítica (§5).

1. Os 6 casos de teste

Cada caso usa as 6 features e as faixas `baixo/medio/alto` do `CLAUDE.md`. Os perfis são os mesmos da versão anterior deste relatório — o que mudou foi apenas a massa de treino sobre a qual eles são classificados.

Caso	Propósito	complexidade	loc	n_autores	churn	n_imports	cobertura	Intuição
a	Baixo risco claro — tudo bom	baixo	medio	baixo	baixo	baixo	alto	BAIXO
b	Alto risco claro — tudo ruim	alto	alto	alto	alto	alto	baixo	ALTO
c	Ambíguo — sinais contraditórios (complexidade alta e cobertura alta)	alto	medio	medio	medio	medio	alto	~50/50
d	Armadilha de Koru, sem bônus de interação — arquivo pequeno e denso	alto	baixo	medio	medio	medio	medio	ALTO
e	Combinação rara / fronteira — arquivo grande porém simples, churn alto	baixo	alto	baixo	alto	baixo	alto	BAIXO/ indefinido
f	Feature contestada — tudo ruim, mas cobertura alta	alto	alto	alto	alto	alto	alto	ALTO

2. Resultados

Caso	P(SIM)	P(NAO)	Recomendação	Bate com a intuição?
a	4,41%	95,59%	BAIXO RISCO	Sim
b	95,26%	4,74%	ALTO RISCO	Sim
c	49,08%	50,92%	BAIXO RISCO	Sim, e de forma mais literal — cai a poucas décimas de 50/50, a marca mais nítida de "caso ambíguo" das três versões deste relatório (ver discussão abaixo)
d	67,26%	32,74%	ALTO RISCO	Sim, com folga maior que na versão anterior — sem bônus de interação, ainda assim mais decisivo que o 55,5 % da versão com N=150 (ver discussão abaixo)
e	23,46%	76,54%	BAIXO RISCO	Sim (com ressalva sobre confiança — ver §4.3)
f	92,83%	7,17%	ALTO RISCO	Sim — cobertura alta reduz um pouco o risco frente ao caso b, mas não o suficiente para "salvar" o módulo

As duas probabilidades somam 100,00 % em todos os casos e as 6 features casaram com a tabela de verossimilhanças ($n_features = 6/6$).

Como cada resultado se forma

O classificador é, na prática, um **somatório de log-odds**: começa no log-odds a priori $\ln(350.000/650.000) = -0,619$ (a base já pende para NAO) e soma o log-odds de cada categoria observada (tabela do §3). $P(SIM) > 50 \% \Leftrightarrow$ soma total > 0 .

- **Caso a** — soma das features = $-2,46$; com o prior, total $-3,08 \rightarrow P(SIM) 4,4 \%$. Todas as categorias "boas" têm log-odds negativo; o modelo acumula evidência contra defeito. Correto.
- **Caso b** — soma das features = $+3,62$; total $+3,00 \rightarrow P(SIM) 95,3 \%$. Espelho do caso a. Correto.
- **Caso c** — soma das features = $+0,58$; total $-0,04 \rightarrow P(SIM) 49,1 \%$. A **complexidade = alto** sozinha vale **+0,98**, mas **n_autores = medio** ($-0,12$), **n_imports = medio** ($-0,09$), **loc = medio** ($-0,21$) e **cobertura = alto** ($-0,22$) puxam na direção contrária — desta vez quase exatamente o suficiente para empatar o placar. Na versão com N=150 este mesmo perfil dava BAIXO RISCO com 38,0 % de folga; com N=1.000.000 dá também BAIXO RISCO, mas a **1 ponto percentual** de 50/50. Não é um erro: com uma amostra de treino $6.667\times$ maior, os log-odds de cada categoria convergem para valores mais próximos do "verdadeiro" peso implícito no gerador (§4.2), e por acaso — não por desenho — esse perfil específico fica quase exatamente no ponto de equilíbrio entre as evidências a favor e contra. Continua sendo, como o nome propõe, o mais perto de 50/50 dos seis.
- **Caso d** — soma das features = $+1,34$; total $+0,72 \rightarrow P(SIM) 67,3 \%$. Aqui está o ponto de Koru, sem bônus de interação: **loc = baixo** tem log-odds **+0,36** (empurra para SIM, não para NAO) e **complexidade = alto** tem **+0,98** — juntas somam $+1,34$, e as outras 4 features (todas **medio**) somam $-0,00$ (ficam quase neutras entre si), resultando no total acima. O modelo classifica o arquivo pequeno-e-denso como ALTO RISCO com folga bem maior que na versão com N=150 (67,3 % vs. 55,5 %) — não por ter reintroduzido nenhum bônus de interação (não há: o SQL continua somando 6 termos independentes), mas porque, com mais dados, os log-odds individuais de **complexidade = alto** e **loc = baixo** ficaram mais próximos dos pesos reais do gerador ($PESOS["cc"]["alto"] = 0,75$ e $PESOS["loc"]["baixo"] = 0,45$) do que as estimativas ruidosas de N=150. Segue muito abaixo dos 74,4 % da versão original (que incluía um bônus de interação plantado de propósito no gerador antigo). Discussão completa em §4.1.
- **Caso e** — soma das features = $-0,56$; total $-1,18 \rightarrow P(SIM) 23,5 \%$. O **churn = alto** ($+1,15$) puxa para SIM, mas é superado por **complexidade = baixo** ($-0,48$), **n_autores = baixo** ($-0,72$), **n_imports = baixo** ($-0,35$) e **cobertura = alto** ($-0,22$). Resultado: BAIXO RISCO. Discussão em §4.3.
- **Caso f** — soma das features = $+3,18$; total $+2,56 \rightarrow P(SIM) 92,8 \%$. É o caso b com **cobertura** trocada de **baixo** para **alto**. O resultado **caiu** de 95,26 % para 92,83 %: no treino atual, **cobertura = baixo** tem log-odds $+0,22$ e **cobertura = alto** tem $-0,22$ — cobertura se comporta na di-

reção "intuitiva" (mais cobertura, menos risco), como já vinha se comportando na versão com N=150. Mesmo assim, o módulo continua ALTO RISCO com folga: a diferença entre os dois casos ($\approx 0,44$ de log-odds) é pequena perto da soma das outras 5 features ruins ($\approx +3,18$). Ver §4.4 e §5.

3. Poder discriminativo das features (log-odds)

Log-odds de uma categoria = $\ln(P(\text{categoria} \mid \text{SIM}) / P(\text{categoria} \mid \text{NAO}))$, o "peso de evidência" daquela categoria: positivo empurra a classificação para SIM, negativo para NAO, e |valor| grande = categoria informativa.

Log-odds por categoria

Feature	baixo	medio	alto	Formato
n_autores	−0,72	−0,12	+0,93	monótono forte
complexidade	−0,48	+0,32	+0,98	monótono
churn	−0,48	+0,24	+1,15	monótono, salto grande no alto
loc	+0,36	−0,21	+0,05	em U (baixo e alto arriscados, mediano protege)
cobertura	+0,22	−0,03	−0,22	monótono fraco (direção "intuitiva")
n_imports	−0,35	−0,09	+0,29	monótono

Ranking por |log-odds| médio (poder discriminativo)

#	Feature	log-odds médio	log-odds máx
1	churn relativo	0,625	1,152
2	nº de autores	0,592	0,930
3	complexidade ciclomática	0,591	0,978
4	nº de imports	0,241	0,347
5	LOC	0,207	0,362
6	cobertura de testes	0,157	0,223

Note-se que o ranking **mudou de ordem novamente** em relação à versão anterior deste relatório (lá, com N=150: autores > complexidade > churn > loc > cobertura > imports). Isso não é um sinal de instabilidade do modelo — é o efeito esperado de reduzir o ruído amostral: com N=150, cada log-odds era estimado sobre poucas dezenas de observações por célula (feature × categoria × classe), então a ordem entre features de poder discriminativo parecido podia trocar por acaso. Com N=1.000.000, os três primeiros lugares (churn, autores, complexidade) seguem próximos entre si (0,59–0,63) porque os pesos programados no gerador (**PESOS** no topo de `dados/gerar_dados.py`) têm amplitude parecida para essas três — **cc**: −0,35 a +0,75 (amplitude 1,10); **n_autores**: −0,35 a +0,90 (amplitude 1,25); **churn**: −0,40 a +0,85 (amplitude 1,25) — então qual delas fica em 1º, 2º ou 3º lugar é sensível a detalhes finos da amostra, mesmo com 1 milhão de registros. O mesmo raciocínio explica **nº de imports** ter subido de 6º (última posição, com N=150) para 4º lugar: sua amplitude de peso (−0,15 a +0,35, amplitude 0,50) é na verdade maior que a de LOC (0,00 a +0,45, amplitude 0,45) — só não aparecia assim com N=150 por ruído de amostragem. **Cobertura de testes** continua, nas duas versões, a feature **menos discriminativa** — coerente com sua amplitude de peso programada ser a menor de todas (−0,15 a +0,20, amplitude 0,35) e com a literatura contestada sobre essa métrica.

4. Análise — as 4 perguntas da atividade

4.1 (a) O modelo classificou conforme a intuição do domínio?

Em geral, sim, com um contraste pedagógico importante no caso d. Os dois casos "extremos" (a e b) saíram exatamente como o domínio prevê. Caso a caso:

- **Casos a e b** (baixo/alto risco claros): acertos limpos, 4,4 % e 95,3 % de P(SIM). O modelo empilha evidência coerente.
- **Caso c** (ambíguo): o resultado (49,1 %, BAIXO RISCO) ficou a **1 ponto percentual** de 50/50 — a marca mais nítida de "caso ambíguo" das três versões deste relatório até aqui (era 61 % ALTO com dados correlacionados; 38 % BAIXO com N=150 independente; agora 49,1 % BAIXO com N=1.000.000). A proximidade crescente de 50/50 não é um objetivo de projeto — é uma coincidência de como esse perfil específico interage com os pesos do gerador — mas ilustra bem que "caso ambíguo" significa cair perto da fronteira, não sempre do mesmo lado dela. A mudança de valor de uma versão para outra é consequência de trocar a massa de treino (Etapa 2), não um defeito do classificador.
- **Caso d — a "armadilha de Koru", sem bônus de interação plantado, agora com estimativas mais precisas.** Este é o caso mais importante para entender a diferença entre as versões deste projeto. Na versão original (dados com correlações plantadas), o gerador somava um **bônus explícito** de +0,70 ao score de risco sempre que `loc = baixo` e `complexidade = alto` ocorriam juntas — um efeito de interação plantado de propósito, e o resultado era P(SIM) = 74,40 %. Desde a decisão "Naive Bayes puro" (Etapa 2, §1.2), essa dependência entre features foi removida do gerador: o rótulo continua dependendo de cada feature, mas **não há nenhum termo que dependa da combinação das duas**. O resultado atual (P(SIM) = 67,26 %) é, por construção, exatamente a soma dos dois efeitos individuais — o log-odds de `loc = baixo` isolado (+0,36) mais o log-odds de `complexidade = alto` isolado (+0,98), ajustado pelas outras 4 features (que somam ≈ 0 entre si) e pelo prior — **sem nenhum bônus extra**. Note que esse valor é mais alto do que os 55,5 % obtidos com N=150 no mesmo perfil, embora nenhuma interação tenha sido reintroduzida: com mais dados, os log-odds individuais de `complexidade = alto` e `loc = baixo` convergiram para mais perto dos pesos reais definidos no gerador (`PESOS["cc"]["alto"] = 0,75`, `PESOS["loc"]["baixo"] = 0,45` — bem próximos dos 0,98 e 0,36 observados, já contando o efeito do ruído gaussiano somado ao rótulo). O modelo continua classificando o arquivo pequeno-e-denso como ALTO RISCO com folga bem menor que os 74,4 % da versão com bônus de interação — mas com folga maior que a versão anterior com N=150, porque a estimativa da soma das partes ficou mais precisa, não porque uma interação foi adicionada. Isso demonstra exatamente o que o Naive Bayes faz e não faz: ele **soma evidências marginais**, nunca modela um efeito de interação que exceda a soma das partes — quando não há interação plantada nos dados, o resultado reflete apenas essa soma, sem "descontos" nem "bônus" adicionais, por mais dados que se acrescente.
- **Caso e** (raro): resultado plausível (BAIXO RISCO, 23,5 %), com a ressalva de §4.3 sobre o modelo soar confiante para um perfil nunca visto no treino.
- **Caso f** (cobertura contestada): a cobertura alta reduziu discretamente o risco frente ao caso b (95,26 % → 92,83 %), mas não o suficiente para tirar o módulo de ALTO RISCO — as outras 5 features ruins dominam. A cobertura se comporta na direção "intuitiva" (mais cobertura, menos risco); mesmo assim seu efeito é pequeno frente às demais features, consistente com a literatura que trata essa relação como fraca/contestada (§4.2).

4.2 (b) Quais features tiveram maior log-odds (maior poder discriminativo)?

Ranking do §3:

1. **Churn relativo** lidera desta vez (|log-odds| médio 0,625; a categoria `alto` sozinha vale +1,15, o maior valor máximo isolado da tabela).
2. **Nº de autores distintos** vem logo atrás (0,592; `alto` = +0,93).
3. **Complexidade ciclomática** fecha o pódio, praticamente empatada com autores (0,591; `alto` = +0,98 — a categoria individual mais forte de toda a tabela).

4. **Nº de imports** salta para o 4º lugar (0,241; máximo 0,347) — na versão com N=150 aparecia em último; com N=1.000.000 sua posição real (entre LOC e cobertura) fica mais visível (ver nota sobre ruído amostral no fim de §3).
5. **LOC** é o caso mais interessante estruturalmente: o log-odds é **em U** — **baixo** (+0,36) e **alto** (+0,05) empurram para SIM, **medio** (−0,21) para NAO. O modelo continua capturando a não-linearidade de Koru (§4.1, caso d), e isso não depende de LOC estar correlacionado com nenhuma outra feature — é uma propriedade da relação entre LOC e o rótulo, isolada.
6. **Cobertura de testes** é a **menos discriminativa** de todas (|log-odds| médio 0,157; máximo 0,223), com efeito pequeno e na direção "intuitiva" — ainda assim, pequeno demais para mudar uma classificação por conta própria (caso f). Consistente com a literatura que trata a relação cobertura↔defeitos como contestada (Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017), e com ela ter a menor amplitude de peso programada no gerador entre as 6 features.

O fato de o ranking ter mudado de ordem **novamente** frente à versão com N=150 (lá: autores > complexidade > churn > loc > cobertura > imports) é esperado — mas por um motivo diferente do que explicava a mudança de ordem na transição anterior (dados correlacionados → independentes). Aqui as features já eram geradas de forma independente nas duas versões comparadas; o que mudou foi só o tamanho da amostra, então a explicação é puramente **estatística**: com N=150, os log-odds de features de poder discriminativo parecido (churn, autores, complexidade — amplitudes de peso programadas entre 1,10 e 1,25) e de features mais fracas mas próximas entre si (LOC e imports — amplitudes 0,45 e 0,50) estavam sujeitos a ruído de amostragem grande o suficiente para trocar a ordem entre pares próximos. Com N=1.000.000 esse ruído cai (erro-padrão $\propto 1/\sqrt{N}$ — o mesmo princípio da Etapa 2, §2.2), e a ordem observada passa a refletir com mais fidelidade a amplitude de peso programada no gerador. Cobertura de testes, cuja amplitude de peso é a menor de todas por desenho, permanece a menos discriminativa nas duas versões — esse ponto não mudou porque a distância dela para as outras features é grande o bastante para não depender do tamanho da amostra.

4.3 (c) O caso de valor não visto / na fronteira (caso e) e o papel de Laplace

O perfil do caso e (**complexidade baixo, loc alto, n_autores baixo, churn alto, n_imports baixo, cobertura alto**) é raro, mas — diferente das versões anteriores deste relatório, com N=150 — não é mais totalmente ausente do treino: aparece em **47 dos 1.000.000 de módulos** (**perfil exato no treino = 47**, contra 0/150 antes). Isso já é, por si só, um efeito interessante de aumentar N: um perfil de 6 categorias tem $3^6 = 729$ combinações possíveis, e com 150 exemplos a chance de qualquer combinação específica aparecer nem uma vez era alta; com 1.000.000 de exemplos, mesmo perfis de baixa probabilidade conjunta (se as 6 categorias fossem independentes entre si dentro de cada classe, o que elas são, por desenho — §1.2 da Etapa 2) passam a aparecer algumas dezenas de vezes. Ainda assim, 47/1.000.000 é uma fração pequena o bastante para o ponto pedagógico original continuar valendo. O classificador devolve uma resposta bem definida: $P(\text{NAO}) = 76,5\%$.

Por que ele consegue responder mesmo quando o perfil é raro. O Naive Bayes nunca precisa ter visto o perfil completo — pela hipótese de independência, ele só usa as **seis fatias unidimensionais** $P(\text{categoria} \mid \text{classe})$, e cada uma dessas fatias está bem povoada (a menos população das 36 combinações feature×categoria×classe tem quase 30 mil observações — ver adiante). Ou seja, a esparsidade da combinação de 6 categorias não trava o modelo mesmo quando ela é rara ou ausente. Essa é a **força** do Naive Bayes — e também o seu risco: ele soa confiante (77/23) sobre uma combinação que talvez mereça mais incerteza justamente por ser rara no treino, mesmo com N grande.

O que a suavização de Laplace faz aqui. Com este dataset, a célula mais rara de todas (das 36 combinações feature × categoria × classe) é **churn = alto** na classe NAO: 29.682 dos 650.000 módulos sem defeito — ainda assim, uma contagem grande em termos absolutos. Sem suavização, $P(\text{churn=alto} \mid \text{NAO})$ seria $29.682/650.000 \approx 0,045665$; com Laplace, $(29.682+1)/(650.000+3) = 29.683/650.003 \approx 0,045666$ — a diferença na quarta casa decimal já é praticamente irrelevante. Isso é esperado: com N=150, a correção de Laplace (+1 no numerador, +3 no denominador) tinha um efeito proporcional visível mesmo nas células mais povoadas; com N=1.000.000, a mesma cor-

reção fixa (+1, +3) é desprezível frente a qualquer contagem observada — Laplace continua presente e correto, só que seu efeito prático encolheu ainda mais com o aumento de N.

Quando Laplace seria decisivo. Se tivéssemos discretizado em mais faixas, ou combinado features raras o bastante para zerar alguma célula, essa célula ficaria em 0 e — sem suavização — $\ln(0) = -\infty$ zeraria a classe inteira por causa de **uma** categoria não observada. Mesmo com $N=1.000.000$, esse risco não desaparece por completo (basta discretizar mais fino ou reduzir o tamanho de uma subpopulação), só fica cada vez menos provável de ser acionado nesta configuração específica de 6 features $\times$ 3 categorias. É uma apólice de seguro que, neste conjunto de dados, segue sem precisar ser acionada — mas que mantém o modelo bem-definido para qualquer entrada, independente do tamanho de N.

4.4 (d) Limitações do Naive Bayes neste domínio

1. **A independência entre features testada aqui é verdadeira por construção, não por realismo.** Esta é a limitação central desta versão do projeto, igual à das duas versões anteriores — o que mudou foi só o quanto os dados a confirmam. Os dados de treino da Etapa 2 foram gerados com as 6 features estatisticamente independentes entre si (decisão "Naive Bayes puro" — CLAUDE.md), e a matriz de correlação da Etapa 2 confirma isso com folga maior do que antes: todos os $|r| \leq 0,002$ (contra $|r| < 0,10$ com $N=150$), bem dentro do erro-padrão esperado de correlação nula nesta escala ($\approx 1/\sqrt{N-2} \approx 0,001$). Isso significa que **não há**, neste dataset, nenhuma violação empírica de independência para medir — e a análise abaixo não afirma ter encontrado uma; o aumento de N só reforça estatisticamente essa afirmação, não a muda. O que existe é uma limitação **teórica, documentada na literatura**: em código real, complexidade ciclomática e LOC são fortemente correlacionadas ($R^2 \approx 0,93$ — Shepperd, 1988; Seção 5 do relatório da Etapa 1). Um classificador treinado sobre dados sintéticos independentes se comporta exatamente como a teoria do Naive Bayes prevê (soma de evidências marginais, sem "contar duas vezes"), mas isso não garante o mesmo comportamento sobre uma massa de dados real, onde essa correlação existiria de fato. A distinção importa: é uma limitação **do realismo da simulação**, não um erro encontrado nos dados nem um erro de implementação do classificador — e nenhuma quantidade de dados sintéticos adicionais muda isso, porque a limitação está na forma como os dados são gerados, não no tamanho da amostra.
2. **O Naive Bayes não modela interação entre features — nem para mais, nem para menos.** O caso d (§4.1) evidencia isso de forma limpa: sem nenhum termo de interação nos dados, o resultado para "pequeno e denso" é exatamente a soma dos dois efeitos individuais (LOC baixo + complexidade alta), nem mais nem menos. Se, em um cenário real, o efeito combinado de "pequeno e denso" fosse **maior** do que a soma das partes (o que a teoria de Kuru et al. sugere ser plausível), o Naive Bayes subestimaria esse risco — não porque os dados de treino "escondam" a interação, mas porque o modelo, por definição, não tem como representar termos de interação.
3. **Relação não-linear de LOC com risco.** Isso continua valendo e **não** depende de LOC estar correlacionado com outra feature: a discretização em 3 faixas captura a forma em U (§3) porque o peso atribuído a `loc = baixo` na geração dos dados é individualmente maior que o de `loc = medio` — uma propriedade de LOC isolado, coerente com Kuru et al. (2008).
4. **Feature de evidência contestada: cobertura de testes.** Continua entre as duas features menos discriminativas do ranking (§3), com efeito pequeno sobre o score final (caso f). Mantê-la é uma limitação assumida e documentada, não um descuido.
5. **Simplificação módulo = arquivo (e não classe).** Inalterada frente à versão anterior: o estudo de referência (Kuru et al.) usa classe como unidade; adotamos arquivo por ser universal e não exigir parser por linguagem, ao custo de métricas mais "grossas" por registro.
6. **Dados de treino sintéticos.** Os 1.000.000 de módulos foram gerados por script (Etapa 2). Assim como na versão anterior, o rótulo aqui depende das 6 features de forma independente, mas o rótulo em si ainda é definido por pesos escolhidos pelo autor a partir da literatura — a validação contra a "intuição do domínio" continua, em parte, circular quanto a essa direção individual de cada feature (mesmo não sendo mais circular quanto à interação entre elas).

5. Reflexão Crítica

Este projeto testou o classificador Naive Bayes exatamente no cenário em que a sua própria suposição central — independência condicional entre features dado o rótulo — é **verdadeira por construção dos dados**. Os resultados confirmam que, nesse cenário, o modelo se comporta **exatamente como a teoria prevê**: separa com folga os módulos claramente arriscados dos claramente seguros (casos a e b, 4 % vs 95 %), responde de forma bem definida a um perfil raro no treino porque só depende de fatias unidimensionais bem povoadas (caso e, §4.3), e, no caso mais revelador (caso d), produz exatamente a **soma** dos dois efeitos marginais de "LOC baixo" e "complexidade alta" — nem mais, nem menos — porque não há, nos dados, nenhum termo de interação para o modelo capturar ou deixar de capturar. Passar de $N=150$ para $N=1.000.000$ não mudou nenhuma dessas conclusões qualitativas; mudou os números de segunda casa decimal (§2) e deixou a matriz de correlação da Etapa 2 ainda mais perto de zero (§4.4, item 1) — o que reforça, sem alterar, o argumento a seguir.

Isso é bom e é ruim ao mesmo tempo, e é importante não confundir as duas coisas. É **bom** porque válida a implementação: o classificador SQL soma priors e verossimilhanças em log corretamente, aplica Laplace corretamente e produz decisões coerentes com os pesos que geraram os dados — se houvesse um bug na Etapa 3, este seria o cenário mais fácil de expor um comportamento errado, e não expôs nenhum. É **limitado** porque este é o cenário mais favorável possível para um Naive Bayes, e não é o cenário de um sistema real em produção. Em código real, complexidade ciclomática e LOC são correlacionadas com força ($R^2 \approx 0,93$ — Shepperd, 1988), e o Naive Bayes, ao tratá-las como independentes, contaria a evidência de "tamanho/estrutura do código" duas vezes, ficando mais confiante do que deveria exatamente nos casos onde essas features concordam. Este projeto **não mediu** esse efeito porque optou, de propósito, por não reproduzi-lo nos dados sintéticos (Etapa 2, §1.2) — então é preciso ser preciso sobre o que está sendo afirmado aqui: isto é uma limitação **teórica, documentada na literatura consultada na Etapa 1**, não uma violação **encontrada empiricamente** nesta massa de dados. A matriz de correlação da Etapa 2 saiu com todos os $|r| \leq 0,002$ — uma amostra de 1.000.000 de registros não deixa dúvida estatística de que a correlação populacional entre essas features, nos dados sintéticos, é de fato zero; não há, neste dataset, nada para "descobrir" nesse sentido, nem havia com $N=150$ — só ficou mais difícil de duvidar.

O veredito honesto: o modelo implementado está correto e bem testado dentro do cenário para o qual foi desenhado — um Naive Bayes "em sua forma pura". Antes de usá-lo sobre dados reais de um repositório de verdade, seria necessário reavaliar a suposição de independência sobre a massa real (medir a correlação entre complexidade e LOC nesse repositório, por exemplo) e, se ela se confirmar como na literatura, considerar reduzir a redundância (fundir as duas features, ou trocar por um modelo que aceite correlação, como regressão logística) antes de confiar nas probabilidades absolutas — não só na ordenação relativa dos módulos — que o classificador produz.

6. Desempenho

Com N passando de 150 para 1.000.000 de registros (Etapa 2, ver CLAUDE.md), faz sentido medir — não só assumir — que o pipeline continua rodando em segundos, não minutos. Os números abaixo vêm direto dos scripts, que instrumentam e imprimem seu próprio tempo de execução (`time.perf_counter`): `dados/gerar_dados.py` mede a geração dos dados; `sql/rodar_classificador.py` mede a importação do CSV; `testes/rodar_casos_teste.py` mede um único tempo de parede (wall-clock), do início do script até obter os resultados dos 6 casos de teste (essa medição é salva na seção "DESEMPENHO" de `testes/resultados_casos.txt`, gerada a cada execução).

Etapa	O que mede	Tempo
Etapa 2 — geração dos dados	1.000.000 de módulos sintéticos (numpy vetorizado + pandas), ponta a ponta	11,88 s
Etapa 3 — importação do CSV	<code>read_csv_auto</code> → tabela <code>dados_treinamento</code> (1.000.000 de linhas)	3,33 s
Etapa 4 — classificar os 6 casos, ponta a ponta	Conectar ao banco, importar o CSV de treino, criar as views e classificar os 6 casos de teste	4,12 s

Detalhamento interno da Etapa 2 (geração de features / geração do rótulo / escrita do CSV / cálculo das estatísticas de validação) fica em `dados/etapa2_validacao.txt` §(f).

Leitura dos números. O pipeline inteiro continua na casa dos segundos, não minutos — a checagem pedida no CLAUDE.md ao aumentar `N_REGISTROS` passa com folga. A etapa mais lenta é a geração dos dados (Etapa 2); as Etapas 3 e 4 recriam o banco DuckDB do zero (importação do CSV + criação das views + classificação) em poucos segundos, mesmo com `treino_longo` chegando a 6.000.000 de linhas.

Classificar um caso novo não fica mais lento com N maior. A consulta que classifica um perfil de 6 categorias usa só a view `verossimilhancas`, de tamanho **fixo** — 6 features × 3 categorias × 2 classes = 36 linhas — não importa se o treino tem 150 ou 1.000.000 de módulos. O Naive Bayes usa apenas essas 36 fatias unidimensionais de $P(\text{categoria} \mid \text{classe})$ (§4.3), nunca uma busca sobre `dados_treinamento` inteiro; por isso o custo de classificar um caso não escala com N.

Nota sobre o uso de IA

Os casos de teste, o script de execução e esta análise foram construídos em diálogo com uma IA generativa. Cada resultado numérico foi reproduzido pelo script (`testes/rodar_casos_teste.py`) e conferido contra a decomposição em log-odds; as limitações teóricas foram checadas contra as referências levantadas na Etapa 1. A tese central desta versão do relatório — a distinção entre "limitação teórica documentada na literatura" e "violação encontrada empiricamente nos dados" — foi definida como decisão de projeto ("Naive Bayes puro", CLAUDE.md) antes da geração dos dados, não inferida depois de olhar os resultados. Todo o conteúdo é defensável oralmente.